

ELO329 - Diseño y Programación Orientados a Objetos

Clases en C++

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

Ya conocemos la idea de clases y objetos desde Java. El objetivo ahora no es aprender POO desde cero, sino entender cómo C++ implementa esas mismas ideas.

La diferencia principal es que C++ obliga a ser más explícitos en tres decisiones:

- Dónde vive el objeto.
- Quién puede modificar su estado.
- Cuándo se construye y cuándo se destruye.

La idea orientada a objetos es la misma. Lo nuevo es la semántica de C++.

Contenidos

1. Clases y objetos en C++
2. Objetos en stack, heap y segmentos de datos
3. Archivos `.h` y `.cpp`
4. Calificadores de acceso
5. Constructores y destructores
6. Métodos `const`, sobrecarga y listas de inicialización

Clases y objetos

Una clase describe el estado y el comportamiento de un tipo de objeto.

Un objeto es una instancia concreta de esa clase.

```
class Point {  
public:  
    void draw();  
  
private:  
    int m_X;  
    int m_Y;  
};
```

En C++, los atributos suelen llamarse miembros dato y los métodos suelen llamarse funciones miembro.

Primero entendemos qué es una clase. Luego veremos qué ocurre cuando esa clase se convierte en un objeto real en memoria.

Terminología habitual

Concepto en Java	Término frecuente en C++	Idea
Atributo	Miembro dato	Estado interno del objeto
Método	Función miembro	Operación asociada al objeto
Objeto	Instancia	Valor concreto de una clase
Visibilidad	Calificador de acceso	Control de uso desde fuera

El vocabulario cambia, pero el modelo conceptual sigue siendo el mismo.

Diferencia central con Java

Java

- Una variable de clase almacena una referencia.
- El objeto se crea con `new`.
- El objeto queda en el heap.

C++

- Una variable puede ser el objeto mismo.
- El objeto puede crearse sin `new`.
- Su vida depende de dónde se declara.

Este cambio explica gran parte de la sintaxis que veremos después: constructores, destructores, punteros, referencias y copias.

Dónde puede vivir un objeto

Ubicación	Cómo aparece	Cuándo se destruye
Stack o memoria automática	Variable local	Al salir del bloque
Heap o memoria dinámica	<code>new</code>	Al usar <code>delete</code>
Segmentos de datos	Variable global o estática	Al terminar el programa

Esta diferencia afecta la forma de copiar, pasar y destruir objetos.

Antes de escribir una clase completa, necesitamos reconocer qué tipo de objeto estamos creando.

Creación de objetos

```
Persona juan;           // objeto local
Persona maria("Maria"); // objeto local con constructor

Persona *pJuan = new Persona("Juan"); // objeto en el heap

Persona &rJuan = juan; // referencia, es un alias
```

Lectura:

- `juan` y `maria` son objetos.
- `pJuan` es un puntero a un objeto.
- `rJuan` no es un objeto nuevo, es otro nombre para `juan`.

Estas formas no son equivalentes. Cambian la duración del objeto y también la forma de acceder a sus miembros.

Acceso a objetos

Una vez creado el objeto, la siguiente pregunta es cómo se invocan sus operaciones.

Si tenemos el objeto directamente, usamos `.`.

```
juan.saludar();
```

Si tenemos un puntero al objeto, usamos `->`.

```
pJuan->saludar();
```

El segundo caso se parece más a Java en la forma de pensar, porque se trabaja a través de una referencia indirecta.

Ahora que sabemos crear y acceder a objetos, pasamos a cómo organizar el código de una clase en C++.

Separación de archivos

Cuando una clase crece, mezclar definición e implementación vuelve difícil leer el programa.

Por eso en C++ es común separar dos responsabilidades.

Archivo	Qué contiene	Rol
Clase.h	Definición de la clase	Interfaz
Clase.cpp	Implementación de métodos	Código ejecutable
main.cpp	Uso de las clases	Programa

El archivo `.h` dice qué se puede usar. El archivo `.cpp` dice cómo se hace.

Esta separación anticipa una idea importante: una clase debe mostrar una interfaz clara y ocultar sus detalles internos.

Organización recomendada

```
Point.h  
Point.cpp  
  
Automobile.h  
Automobile.cpp  
main.cpp
```

Se puede poner más de una clase por archivo, pero normalmente conviene separar una clase por encabezado e implementación.

Esta organización facilita leer, compilar y mantener el programa.

Estilo Java

En Java es habitual que la clase completa quede en un archivo. La definición de atributos y la implementación de métodos aparecen juntas.

```
class Employee {  
    public Employee(String n, double s) {  
        name = n;  
        salary = s;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    private final String name;  
    private double salary;  
}
```

Estilo C++: encabezado

En C++ el encabezado muestra la interfaz de la clase. Quien use `Employee` necesita saber qué constructores y métodos existen, pero no necesariamente cómo están implementados.

```
#include <string>

class Employee {
public:
    Employee(std::string n, double s);
    std::string getName() const;
    double getSalary() const;
    void raiseSalary(double byPercent);

private:
    const std::string name;
    double salary;
};
```

Estilo C++: implementación

En el archivo `.cpp` se escriben los cuerpos de las funciones. Aquí aparece el detalle que el usuario de la clase no necesita ver de inmediato.

```
#include "Employee.h"

Employee::Employee(std::string n, double s)
    : name(n), salary(s) {

std::string Employee::getName() const {
    return name;
}

double Employee::getSalary() const {
    return salary;
}
```

Estilo C++: implementación

`Employee::getName` significa que `getName` pertenece a `Employee`.

Con esta separación clara, podemos mirar una clase pequeña y distinguir su interfaz de su estado interno.

Ejemplo: Point.h

`Point.h` muestra lo mínimo que necesitamos para describir un punto: sus operaciones públicas y sus coordenadas privadas.

```
class Point {  
public:  
    Point();  
    void draw();  
    void moveTo(int x, int y);  
    void lineTo(int x, int y);  
  
private:  
    int m_X;  
    int m_Y;  
};
```


Ejemplo: Point.h

Puntos importantes:

- `public` marca lo visible desde fuera.
- `private` marca el estado interno.
- La definición de la clase termina con el carácter `;`.

La separación entre lo visible y lo oculto nos lleva directamente al tema de visibilidad.

Visibilidad

La visibilidad responde una pregunta simple: ¿quién puede usar cada miembro de la clase?

Calificador	Significado
<code>public</code>	Puede usarse desde fuera de la clase
<code>private</code>	Solo puede usarse dentro de la clase
<code>protected</code>	Puede usarse dentro de la clase y clases derivadas

Reglas por omisión:

- En `class` , lo no marcado es `private` .
- En `struct` , lo no marcado es `public` .

Acceso permitido

Calificador	Misma clase	friend	Derivadas	Otros
private o ausente	Sí	Sí	No	No
protected	Sí	Sí	Sí	No
public	Sí	Sí	Sí	Sí

El encapsulamiento consiste en exponer operaciones y ocultar detalles internos.

La tabla no solo es una regla de sintaxis. Es una herramienta de diseño: permite proteger invariantes del objeto.

Ejemplo: Automobile

Aplicaremos las reglas anteriores a una clase más completa. Queremos modelar un automóvil.

Atributos

- Marca.
- Número de puertas.
- Número de cilindros.
- Tamaño del motor.

Operaciones

- Ingresar datos.
- Mostrar datos.
- Fijar puertas.
- Obtener puertas.

La interfaz pública debe permitir usar el automóvil sin exponer directamente sus atributos.

Automobile.h

El encabezado muestra qué puede hacer un `Automobile` y qué datos guarda internamente.

```
#include <string>

class Automobile {
public:
    Automobile();
    void input();
    void set_NumDoors(int doors);
    void display() const;
    int get_NumDoors() const;

private:
    std::string make;
    int numDoors;
    int numCylinders;
    int engineSize;
};
```

Tipos de funciones miembro

Una vez definida la interfaz, conviene clasificar sus funciones. Esto ayuda a leer la clase y a reconocer responsabilidades.

Tipo	Qué hace	Ejemplo
Accesor	Lee el estado del objeto	<code>get_NumDoors</code>
Mutador	Cambia el estado del objeto	<code>set_NumDoors</code>
Constructor	Inicializa el objeto al crearlo	<code>Automobile</code>
Destructor	Se ejecuta al destruir el objeto	<code>~Automobile</code>

Un buen diseño mantiene el estado privado y entrega acceso controlado mediante métodos públicos.

Los constructores y destructores son especiales porque no representan una operación cualquiera. Representan el inicio y el término de la vida del objeto.

Constructor y destructor

```
class Automobile {  
public:  
    Automobile();           // constructor  
    void input();           // mutador  
    void set_NumDoors(int doors); // mutador  
    void display() const;   // accesor  
    int get_NumDoors() const; // accesor  
    ~Automobile();         // destructor  
  
private:  
    std::string make;  
    int numDoors;  
    int numCylinders;  
    int engineSize;  
};
```

El constructor no declara tipo de retorno. El destructor tampoco.

Ambos están ligados a la ubicación del objeto en memoria.

Creando y usando un objeto

Este `main` muestra la relación entre el usuario de la clase y su interfaz pública.

```
#include "Automobile.h"
#include <iostream>

int main() {
    Automobile myCar;

    myCar.set_NumDoors(4);
    std::cout << "Enter all data for an automobile: ";
    myCar.input();

    std::cout << "This is what you entered: ";
    myCar.display();

    std::cout << "This car has "
              << myCar.get_NumDoors()
              << " doors.\n";
}
```


Lectura del ejemplo

- `Automobile myCar` crea el objeto.
- `myCar.set_NumDoors(4)` llama un método público.
- `myCar.input()` modifica el estado del objeto.
- `myCar.display()` muestra su estado sin cambiarlo.
- `std::cout` es un objeto de salida estándar.

```
std::cout << "saludo";
```

El operador `<<` está definido para escribir datos en el flujo.

El ejemplo muestra dos ideas conectadas: el objeto se crea automáticamente y luego se manipula solo mediante su interfaz pública.

Constructores

Un constructor existe para dejar el objeto en un estado inicial válido.

Se ejecuta automáticamente al crear el objeto.

```
Automobile myCar;
```

En ese punto se invoca:

```
Automobile::Automobile()
```

Si el objeto es global, el constructor corre antes de `main`. Si el objeto es local, corre al entrar al bloque donde se declara.

Por eso no basta con declarar atributos. También necesitamos decidir cómo comienzan sus valores.

Constructor por defecto

Un constructor por defecto no recibe argumentos. Es útil cuando queremos crear un objeto sin entregar datos iniciales desde fuera.

```
Point p;
```

Si no se declara ningún constructor, C++ puede generar uno automáticamente.

Si la clase declara constructores con parámetros, conviene declarar explícitamente el constructor por defecto cuando también se necesite.

Este caso aparece, por ejemplo, al crear arreglos de objetos.

Arreglos de objetos

```
Point drawing[50];
```

En C++ esto crea 50 objetos `Point` .

Cada elemento invoca el constructor por defecto.

En Java, un arreglo de objetos normalmente crea referencias. Los objetos referidos deben crearse aparte.

Esta diferencia vuelve importante que el constructor por defecto exista cuando la clase se usará en arreglos.

Constructor inline

Un método pequeño puede implementarse dentro de la clase. Esto deja la implementación junto a la declaración.

```
class Point {  
public:  
    Point() {  
        m_X = 0;  
        m_Y = 0;  
    }  
  
private:  
    int m_X;  
    int m_Y;  
};
```

Esto se conoce como implementación `inline`.

Cuando el método deja de ser trivial, separar la implementación mejora la lectura.

Funciones out-of-line

Para métodos más grandes se prefiere implementar fuera de la clase.

```
#include "Point.h"

Point::Point() {
    m_X = 0;
    m_Y = 0;
}
```

El operador `::` es el operador de resolución de alcance.

Le dice al compilador que esa función pertenece a la clase `Point`.

Esta notación conecta el `.cpp` con la clase declarada en el `.h`.

Métodos const

Después de separar accedores y mutadores, podemos expresar esa diferencia en el lenguaje.

Un método que no cambia el objeto puede declararse como `const` .

```
class Automobile {  
public:  
    void display() const;  
    int get_NumDoors() const;  
  
private:  
    std::string make;  
    int numDoors;  
    int numCylinders;  
    int engineSize;  
};
```

Métodos const

El compilador ayuda a verificar que estos métodos no modifiquen atributos.

`const` convierte una intención de diseño en una regla verificable por el compilador.

Por qué usar const

```
void Automobile::display() const {  
    std::cout << make << std::endl;  
}
```

La promesa es clara: `display` solo observa el objeto.

Si dentro del método intentamos cambiar un atributo, el compilador reportará error.

Esto reduce cambios accidentales y documenta la intención del método.

Esta idea se vuelve especialmente útil cuando un objeto se comparte mediante referencias o se pasa como argumento.

Constructor de Automobile

Ahora implementamos las operaciones declaradas en `Automobile.h`. Primero, el constructor por defecto.

```
#include "Automobile.h"
#include <iostream>

Automobile::Automobile() {
    numDoors = 0;
    numCylinders = 0;
    engineSize = 0;
}
```

En la implementación no se repite `public` ni `private`.

Esos calificadores pertenecen a la definición escrita en el `.h`.

El constructor deja los enteros en un estado conocido. Sin esta inicialización, sus valores podrían ser basura.

Implementación de display

```
void Automobile::display() const {  
    std::cout << "Make: " << make  
                << ", Doors: " << numDoors  
                << ", Cyl: " << numCylinders  
                << ", Engine: " << engineSize  
                << std::endl;  
}
```

`display` es un accesor. Lee el estado interno y lo envía a la salida estándar.

Por eso se declara como `const`.

Implementación de input

```
void Automobile::input() {  
    std::cout << "Enter the make: ";  
    std::cin >> make;  
  
    std::cout << "How many doors? ";  
    std::cin >> numDoors;  
  
    std::cout << "How many cylinders? ";  
    std::cin >> numCylinders;  
  
    std::cout << "What size engine? ";  
    std::cin >> engineSize;  
}
```

`input` es un mutador porque cambia los atributos del objeto.

Por eso no se declara como `const`.

Sobrecarga de constructores

No siempre queremos construir un objeto vacío. A veces queremos entregarle datos desde el inicio.

Para eso una clase puede tener varios constructores.

```
class Automobile {  
public:  
    Automobile();  
    Automobile(std::string make,  
                int doors,  
                int cylinders,  
                int engineSize = 2);  
    Automobile(const Automobile &a);  
  
    // ...  
};
```

Sobrecarga de constructores

El compilador elige el constructor según los argumentos entregados.

Esta es la misma idea de sobrecarga vista en Java, aplicada al inicio de la vida del objeto.

Argumentos por omisión

```
Automobile(std::string make,  
           int doors,  
           int cylinders,  
           int engineSize = 2);
```

El último argumento puede omitirse.

```
Automobile a("Yugo", 4, 2, 1000);  
Automobile b("Mini", 2, 4);
```

En el segundo caso, `engineSize` toma el valor `2`.

Esto evita duplicar constructores cuando solo cambia un valor común.

Invocación de constructores

```
Automobile myCar;  
  
Automobile yourCar("Yugo", 4, 2, 1000);  
  
Automobile hisCar(yourCar);
```

Lectura:

- `myCar` usa el constructor por defecto.
- `yourCar` usa el constructor con parámetros.
- `hisCar` usa el constructor copia.

Tres formas de crear objetos, tres decisiones distintas sobre el estado inicial.

Constructor con parámetros

El valor por defecto se escribe en la declaración del `.h`, no en la implementación.

```
Automobile::Automobile(std::string p_make,  
                        int doors,  
                        int cylinders,  
                        int engineSize) {  
    make = p_make;  
    numDoors = doors;  
    numCylinders = cylinders;  
    this->engineSize = engineSize;  
}
```

`this` se usa aquí para distinguir el atributo del parámetro.

La implementación cumple el mismo objetivo que el constructor por defecto: dejar el objeto listo para usarse, pero ahora usando datos entregados por quien lo crea.

this

`this` es un puntero al objeto actual.

Si un parámetro tiene el mismo nombre que un atributo, se puede escribir:

```
this->numDoors = numDoors;  
this->numCylinders = numCylinders;
```

Sin `this`, esta asignación sería confusa:

```
numDoors = numDoors;
```

En Java se escribe `this.numDoors`. En C++ se escribe `this->numDoors`.

Esta flecha recuerda que `this` no es el objeto directo, sino un puntero al objeto actual.

Constructor copia

```
Automobile(const Automobile &a);
```

Lectura de la firma:

- Recibe otro `Automobile`.
- Usa referencia para evitar una copia adicional.
- Usa `const` para prometer que no modificará el objeto recibido.

Este constructor permite crear un objeto a partir de otro.

```
Automobile copy(original);
```

La copia es importante en C++ porque las variables pueden contener objetos completos, no solo referencias.

Lista de inicialización

La lista de inicialización permite construir los atributos antes de entrar al cuerpo del constructor.

```
Automobile::Automobile(std::string make,  
                        int doors,  
                        int cylinders,  
                        int engineSize)  
: make(make),  
  numDoors(doors),  
  numCylinders(cylinders),  
  engineSize(engineSize) {  
}
```

Es la forma correcta de inicializar atributos `const`, referencias y objetos miembro.

Conceptualmente, es más precisa que asignar dentro del cuerpo: primero se construyen los miembros, luego se ejecutan las instrucciones del constructor.

Destructores

Si el constructor marca el inicio de la vida del objeto, el destructor marca su término.

Un destructor se ejecuta cuando el objeto deja de existir.

```
Automobile::~~Automobile() {  
}
```

Reglas básicas:

- No recibe parámetros.
- No declara tipo de retorno.
- Se llama igual que la clase, con `~` al inicio.

Su uso es especialmente importante cuando la clase administra recursos como memoria dinámica, archivos o conexiones.

Cuándo se llama el destructor

```
void f() {  
    Automobile car;  
} // aquí se destruye car
```

Para objetos creados con `new`, se debe usar `delete`.

```
Automobile *car = new Automobile();  
  
delete car;
```

En Java, el recolector de basura se encarga de liberar memoria. En C++, la vida del objeto debe entenderse con más precisión.

Esta es la conexión final con el comienzo de la clase: saber dónde vive el objeto permite saber cuándo se destruye.

Ideas clave

- La idea de clase y objeto es la misma que en Java.
- En C++, una variable puede ser el objeto mismo.
- La ubicación del objeto determina su duración.
- La definición de la clase suele ir en `.h`.
- La implementación suele ir en `.cpp`.
- `public`, `private` y `protected` controlan el acceso.
- Constructores y destructores manejan el inicio y término de la vida del objeto.
- `const`, `this` y las listas de inicialización ayudan a escribir clases más claras.